Probabilidade Computacional – Exercício 3

1) Em simulação estocástica, nos interessa ter uma sequência de números pseudoaleatórios distribuídos de forma uniforme entre 0 e 1.

Um possível algoritmo que pode ser utilizado para gerar a sequência desejada é o LCG e cuja relação de recorrência é apresentada a seguir:

$x_k = (a \times x_{k-1} + c)\ mod\ M$

$u_k = x_k / M$

Outro possível método para geração de números pseudoaleatórios, chamado aqui de algoritmo LM, toma como base o lançamento de uma moeda comum que pode gerar dois eventos, cara ou coroa, cada um com probabilidade $0{,}5$. Transformando estes eventos em números binários sendo cara = 0 e coroa = 1, podemos, assim, gerar números decimais a partir de números binários segundo a formulação abaixo usando os lançamentos de uma moeda para definir os coeficientes a_i:

$$Número = a_n 2^{(n-1)} + \cdots + a_4 2^3 + a_3 2^2 + a_2 2^1 + a_1 2^0$$

Por exemplo, se $n = 5$, o que equivale a fazer 5 lançamentos de uma moeda, podemos criar 32 números decimais de 0 a $(2^5 - 1) = 31$. Supondo um número binário de 5 dígitos $(a_5 a_4 a_3 a_2 a_1) = 10011$, obtido a partir de 5 lançamentos de uma moeda, obtemos o número decimal:

$$Número = 1x2^4 + 0x2^3 + 0x2^2 + 1x2^1 + 1x2^0$$

$$Número = 1x16 + 0x8 + 0x4 + 1x2 + 1x1 = 19$$

Se dividirmos o número obtido 19 pelo maior número possível 31, geramos o valor $0{,}6129$ que consequentemente está no intervalo $[0, 1]$. Assim, a partir deste processo, é possível gerar números pseudoaleatórios com distribuição uniforme no intervalo $[0, 1]$.

Um algoritmo mais robusto que os apresentados acima é o Mersenne Twister que fornece uma geração rápida com alta qualidade de aleatoriedade. A partir do que foi exposto e usando os conceitos de simulação, faça:

a) Construa um código baseado no LCG para realizar uma simulação com 10.000 lançamentos simultâneos de duas moedas, ambas honestas, e apresentar em quantos lançamentos o resultado obtido foi **cara** e **coroa**. O LCG deve ser utilizado para tratar o lançamento de ambas as moedas.

b) Construa um código baseado no LCG e no Mersenne Twister para realizar uma simulação com 10.000 lançamentos simultâneos de uma moeda e de um dado (8 faces), ambos honestos, e apresentar em quantos lançamentos o resultado obtido foi simultaneamente **coroa** e **face 5**. O LCG deve ser utilizado para tratar o lançamento da moeda e o Mersenne Twister para tratar o lançamento do dado.

c) Construa um código baseado no LCG e no LM (com lançamento de **10** moedas) para realizar uma simulação com 10.000 lançamentos simultâneos de uma moeda honesta e de um dado (8 faces) viciado (probabilidade de obter a **face 3** é zero e é a mesma para as demais faces) e apresentar em quantos lançamentos o resultado obtido foi simultaneamente **coroa** e **face 8**. O LCG deve ser utilizado para tratar o lançamento da moeda e o LM para tratar o lançamento do dado.

d) Construa um código baseado no Mersenne Twister e no LM (com lançamento de **10** moedas) para realizar uma simulação com 10.000 lançamentos simultâneos de uma moeda viciada (probabilidade de tirar **cara** é **0,45**) e de um dado (**8** faces) honesto, e apresentar a probabilidade de se obter pelo menos um dos resultados: simultaneamente **cara** e **face 1**; simultaneamente **cara** e **face 4**; simultaneamente **coroa** e **face 7**. O Mersenne Twister deve ser utilizado para tratar o lançamento da moeda e o LM para tratar o lançamento do dado.

2) Em um projeto de agricultura de precisão, uma fazenda está testando o uso de drones para lançar sensores de umidade do solo sobre talhões experimentais quadrados de 1 hectare (lado de 100 m). O mecanismo de lançamento possui controles e dispositivos que garantem que os sensores pousarão dentro do talhão—ou seja, a posição de pouso é aleatória dentro do talhão, mas não fora dele. A imprecisão remanescente (vento, turbulência) justifica modelar cada posição como aproximadamente uniforme dentro do talhão e independente entre sensores.

Considere que os pontos $A(x_1, y_1)$, $B(x_2, y_2)$ e $C(x_3, y_3)$ representam as posições onde três sensores caíram, a área do triângulo formado por eles pode ser obtida pela expressão:

$$Área = \frac{1}{2}|x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)|$$

Considere também que, quanto maior a área de cobertura formada pelos três sensores, melhor é o nível de cobertura espacial, cuja qualidade está refletida na tabela ao lado em 4 categorias.

Com base no que foi exposto, estime por simulação, usando o gerador de números aleatórios padrão do R ou Python e semente **123**, a probabilidade de ocorrência de cada categoria de qualidade de cobertura (Ruim, Média, Boa, Excelente).

Qualidade da cobertura	**Proporção da área coberta**
Ruim	Proporção < 0,05
Média	0,05 ≤ Proporção < 0,10
Boa	0,10 ≤ Proporção < 0,20
Excelente	Proporção ≥ 0,20